Practical 1: Uber Fare Prediction Using Linear Regression and Random Forest Regression

1. Aim

To predict the fare of an Uber ride using pickup and drop-off information by implementing Linear Regression and Random Forest Regression, and comparing their performance.

2. Tools and Technologies Used

Python, Pandas, NumPy, Matplotlib, Seaborn and Scikit-learn.

3. Theory / Description

This practical is a supervised machine learning problem based on regression. The objective is to estimate a continuous value - the Uber fare amount - from the available ride information. The dataset contains details such as pickup and drop-off coordinates, passenger count and pickup time. The fare amount is treated as the target variable, while the remaining useful columns are used as input features.

Data Preprocessing and Outlier Detection

Raw data may contain missing values, irrelevant columns and invalid records. Unnecessary columns are removed, missing records are dropped, and the pickup timestamp is converted into a date-time value. The pickup hour is extracted as a numerical feature. Records with invalid fares, passenger counts or geographical coordinates are filtered out.

The Interquartile Range (IQR) method is used to identify unusually high or low fare values. IQR is calculated as the difference between the third quartile (Q3) and first quartile (Q1). Values outside the limits Q1 - 1.5 x IQR and Q3 + 1.5 x IQR are treated as outliers and removed. A boxplot is used to visualize the fare distribution.

Correlation and Regression Models

Correlation analysis measures the relationship between numerical features. A heatmap is used to visualize the correlation values and understand how the input features are related to the fare amount.

Linear Regression is a supervised algorithm that models the relationship between input features and a continuous target using a linear equation. It provides a simple baseline for fare prediction. Random Forest Regression is an ensemble method that builds multiple decision trees and combines their predictions, helping the model capture more complex relationships in the data.

Model Evaluation

The dataset is divided into training and testing sets in an 80:20 ratio. The models are evaluated using the R-squared (R2) score and Root Mean Squared Error (RMSE). R2 indicates how much variation in the fare is explained by the model, while RMSE measures the typical size of prediction errors. A higher R2 and lower RMSE generally indicate better performance.

4. Procedure

1. Import the required libraries and load the Uber dataset using Pandas.
2. Inspect the dataset, its data types, missing values and statistical summary.
3. Remove unnecessary columns and missing records. Convert pickup time into datetime format and extract the pickup hour.
4. Remove invalid fare amounts, passenger counts and geographical coordinates.
5. Calculate the IQR limits for fare amount, remove outliers and display a boxplot.
6. Calculate the correlation matrix and display it using a heatmap.
7. Separate the input features and fare target, then split the data into 80% training and 20% testing sets.
8. Train Linear Regression and Random Forest Regression models, predict test fares, and compare their R2 scores and RMSE values.

5. Working

The notebook first prepares the ride records so that the models are trained on usable values. The pickup hour is retained as a feature, while the original timestamp and identifier columns are removed. IQR filtering reduces the effect of unusually extreme fare values.

After preprocessing, both regression models are trained using the same training data and evaluated on the same test data. Their R2 and RMSE values are printed side by side to support a fair comparison.

6. Applications

- Estimating ride fares before booking.
- Studying how ride distance, location and time relate to fare amounts.
- Supporting transport pricing analysis and service planning.

7. Result

The Uber fare prediction workflow was implemented. The dataset was preprocessed, outliers and correlations were examined, and Linear Regression and Random Forest Regression were trained and compared using R2 and RMSE.

8. Code Explanation

1. The notebook imports Pandas and NumPy for data handling, plotting libraries for visualization, and Scikit-learn for model training and evaluation.
2. The dataset is loaded and cleaned by dropping unused columns and missing values, extracting the pickup hour, and filtering invalid records.
3. Q1, Q3 and IQR are calculated to remove fare outliers. A boxplot and correlation heatmap are generated for data analysis.
4. train_test_split() divides the features and fare target into training and testing sets.
5. LinearRegression() and RandomForestRegressor(n_estimators=10) are trained using fit(). The predict() method generates fare estimates.
6. r2_score() and RMSE calculated from mean_squared_error() are printed to compare the two models.

Practical 2: Email Spam Classification Using KNN and SVM

1. Aim

To classify emails as Not Spam or Spam using the K-Nearest Neighbors and Support Vector Machine algorithms, and analyze their performance.

2. Tools and Technologies Used

Python, Pandas, Matplotlib and Scikit-learn.

3. Theory / Description

Email spam detection is a supervised binary classification problem. Each email is assigned to one of two classes: Normal or Not Spam (0), and Spam (1). The purpose of the model is to learn patterns from previously labelled emails and use those patterns to classify unseen emails.

The dataset represents emails through numerical word-frequency features. The email identifier is not useful for learning the class, so it is removed from the input data. The Prediction column is kept separately as the target variable. The remaining feature columns form the input for the classifiers.

K-Nearest Neighbors (KNN)

KNN is a distance-based classification algorithm. To classify a new email, it calculates the distance between that email and the training records, selects the K nearest records, and assigns the class that occurs most frequently among those neighbours. In this practical, K is set to 5. Since KNN depends on distances between feature values, standardization is applied so that features with larger numerical ranges do not dominate the distance calculation.

Support Vector Machine (SVM)

SVM is a supervised classification algorithm that identifies a decision boundary, called a hyperplane, to separate classes. A linear SVM is used in this practical. It attempts to find a boundary that separates spam and non-spam email feature patterns with a large margin.

Training and Performance Evaluation

The dataset is divided into 80% training data and 20% testing data. Stratified splitting preserves the class proportions in both sets. Both models use a StandardScaler in a Scikit-learn pipeline, ensuring that scaling is fitted on the training data and then applied to the test data.

Performance is analyzed using accuracy, error rate, precision, recall and F1-score. A confusion matrix shows correct and incorrect predictions for each class. These measures provide a broader view than accuracy alone, especially when false spam and missed spam have different consequences.

4. Procedure

1. Import Pandas, Matplotlib and the required Scikit-learn classes and metrics.
2. Load emails.csv and inspect its shape, missing values and class distribution.
3. Remove the email identifier from the input features and separate Prediction as the target.

4. Split the data into training and testing sets using an 80:20 ratio and stratify by the target class.
5. Create a KNN pipeline using StandardScaler and KNeighborsClassifier with K = 5.
6. Create an SVM pipeline using StandardScaler and SVC with a linear kernel.
7. Train both pipelines and predict the classes of the test emails.
8. Calculate the evaluation metrics, display confusion matrices and compare the models.

5. Working

The word-frequency values are used as the numerical representation of each email. KNN classifies an email based on the labels of its five nearest training records, while the linear SVM classifies it according to which side of the learned separating hyperplane it falls on.

The notebook evaluates both predictions against the actual test labels. The confusion matrices and performance metrics help identify how effectively each classifier detects spam and how often it incorrectly classifies normal email.

6. Applications

- Filtering unwanted or suspicious emails.
- Supporting email security and inbox organization.
- Analyzing text-based communication patterns.

7. Result

KNN and linear SVM classifiers were implemented for binary email spam detection. Their predictions were evaluated and compared using classification metrics and confusion matrices.

8. Code Explanation

1. The notebook loads emails.csv, checks the data and separates the input columns from the Prediction target.
2. train_test_split() creates stratified training and testing sets so the class distribution is maintained.
3. Two pipelines are defined: StandardScaler followed by KNeighborsClassifier(K=5), and StandardScaler followed by SVC(kernel="linear").
4. Each pipeline is trained with fit() and used to generate test predictions with predict().
5. Accuracy, error rate, precision, recall and F1-score are calculated for each model. ConfusionMatrixDisplay is used to plot the two confusion matrices.

Practical 3: Bank Customer Churn Prediction Using an Artificial Neural Network

1. Aim

To build a neural network-based classifier that predicts whether a bank customer is likely to leave the bank, and evaluate the model using accuracy and a confusion matrix.

2. Tools and Technologies Used

Python, Pandas, NumPy, Matplotlib, Scikit-learn and TensorFlow/Keras.

3. Theory / Description

Customer churn refers to a situation in which a customer stops using the services of an organization. Predicting churn helps a bank identify customers who may leave and understand where retention efforts may be required. This practical treats churn prediction as a supervised binary classification problem. The target column Exited indicates whether a customer left the bank (1) or stayed (0).

The dataset contains customer-related information such as credit score, geography, gender, age, tenure and account balance. Identifier columns such as RowNumber, CustomerId and Surname are removed because they do not provide useful customer behaviour information for the classifier. Categorical features such as Geography and Gender are converted into numerical columns using one-hot encoding.

Data Preparation and Normalization

The input features and target are separated, and the data is divided into training and testing sets using an 80:20 ratio. StandardScaler is used to normalize the input features. The scaler is fitted on the training data and then applied to both training and testing data. This keeps the test data from influencing the scaling parameters.

Artificial Neural Network

An Artificial Neural Network (ANN) is a machine learning model inspired by the way interconnected neurons process information. It contains an input layer, one or more hidden layers, and an output layer. Each neuron receives values, applies weights and a bias, and passes the result through an activation function.

The practical first builds a network with two hidden Dense layers containing 16 and 8 neurons. ReLU activation is used in the hidden layers, while the output layer contains one neuron with Sigmoid activation. Sigmoid produces a value between 0 and 1, which can be interpreted as the probability of churn.

Model Improvement and Evaluation

The network is compiled with the Adam optimizer and binary cross-entropy loss, which are commonly used for binary classification. A second model is then built with 32 and 16 neurons in its hidden layers and trained for more epochs than the first model. The two models are compared using test accuracy and confusion matrices. The predicted probabilities are converted into class labels using a threshold of 0.5.

4. Procedure

1. Import the required libraries and load Churn_Modelling.csv.
2. Inspect the dataset and remove the RowNumber, CustomerId and Surname columns.
3. Convert Geography and Gender into numerical columns using one-hot encoding.
4. Separate the input features from the Exited target and split the data into 80% training and 20% testing sets.
5. Fit StandardScaler on the training features and transform both training and testing features.
6. Build the first ANN with hidden layers of 16 and 8 neurons and a Sigmoid output layer. Compile and train it for 10 epochs.
7. Build an improved ANN with hidden layers of 32 and 16 neurons and train it for 20 epochs.
8. Predict the test set for both models, convert probabilities to class labels, and print accuracy and confusion matrices.
9. Plot the training and validation accuracy of the improved model.

5. Working

The input customer features pass through the hidden layers, where the network learns combinations of customer characteristics associated with churn. During training, the Adam optimizer updates the network weights to reduce binary cross-entropy loss.

The first network acts as a baseline. The second network uses more neurons and additional training epochs. The notebook compares the test predictions of both networks and plots the improved model's training and validation accuracy over the epochs.

6. Applications

- Identifying customers who may be at risk of leaving a bank.
- Supporting customer retention campaigns and targeted communication.
- Helping financial institutions analyze customer behaviour.

7. Result

A baseline and an improved Artificial Neural Network were built for bank customer churn prediction. Both models were evaluated using accuracy and confusion matrices, and the improved model's accuracy history was visualized.

8. Code Explanation

1. The notebook loads the churn dataset, removes identifier columns and applies one-hot encoding to the categorical columns.
2. The Exited column is selected as the target. The remaining columns are used as input features and are split into training and testing sets.
3. StandardScaler normalizes the input data. The first Sequential model uses Dense layers with 16, 8 and 1 neurons; the second uses 32, 16 and 1 neurons.
4. Both models use ReLU in hidden layers, Sigmoid in the output layer, Adam as optimizer and binary cross-entropy as loss.
5. fit() trains the models. predict() generates probabilities, which are converted to 0 or 1 using the 0.5 threshold.
6. Accuracy and confusion matrices are printed for both models, and the improved model's training and validation accuracy are plotted.

Practical 4: Gradient Descent Algorithm to Find the Minimum of a Function

1. Aim

To implement the Gradient Descent algorithm and find the minimum of the function y = (x + 3)^2, starting from x = 2.

2. Tools and Technologies Used

Python and Matplotlib.

3. Theory / Description

Gradient Descent is an iterative optimization algorithm used to find a minimum of a function. It is widely used in machine learning to adjust model parameters and reduce a loss or error function. The algorithm starts from an initial value and repeatedly updates that value in the direction in which the function decreases most rapidly.

The direction and rate of change are determined by the derivative of the function. For a function f(x), the Gradient Descent update rule is:

Update Rule

x_new = x_old - learning_rate * derivative(x_old)

Here, learning_rate is the learning rate. The learning rate controls the size of each update. A very small learning rate may require many iterations, while a very large learning rate may overshoot the minimum or fail to converge.

Function Used in This Practical

The function considered is y = (x + 3)^2. Its derivative is f'(x) = 2(x + 3). The derivative is zero at x = -3, which is the minimum point of this parabola, and the corresponding function value is 0.

The program starts with x = 2, uses a learning rate of 0.1 and performs 20 iterations. At every iteration, it calculates the derivative and updates x using the Gradient Descent rule. The updated x and function value are displayed. A plot of the function is also generated to show its shape and minimum point.

4. Procedure

1. Import Matplotlib for plotting the function.
2. Define the function f(x) = (x + 3)^2.
3. Define its derivative f'(x) = 2(x + 3).
4. Initialize x = 2, learning_rate = 0.1 and iterations = 20.
5. For each iteration, store the current x value, calculate the derivative and update x using the Gradient Descent formula.
6. Display the iteration number, updated x and corresponding function value.
7. Display the final x value, function value and derivative.
8. Generate a graph of the function over the selected range.

5. Working

At each iteration, the derivative indicates the slope of the function at the current x value. The update subtracts a fraction of this slope from x. Since the starting point is to the right of -3, the updates move x towards the minimum.

After 20 iterations, the value approaches the minimum at x = -3. The graph of the parabola helps visualize why the updates move toward the lowest point.

6. Applications

- Minimizing loss functions during machine learning model training.
- Updating weights and biases in neural networks.
- Solving numerical optimization problems.

7. Result

The Gradient Descent algorithm was implemented for y = (x + 3)^2. Starting from x = 2, repeated updates move the value towards the minimum at x = -3, where the function value is 0.

8. Code Explanation

1. The function(x) method returns (x + 3) squared, while derivative(x) returns 2(x + 3).
2. The program initializes x, the learning rate and the number of iterations. A loop stores the current x value, calculates the next value using the update rule and prints the iteration details.
3. After the loop, the program prints the final x, function value and derivative.
4. The last code cell creates a range of x values, evaluates the function at those points and plots the resulting curve using Matplotlib.

Practical 5: Diabetes Classification Using K-Nearest Neighbors

1. Aim

To implement the K-Nearest Neighbors algorithm on the diabetes dataset and evaluate it using a confusion matrix, accuracy, error rate, precision and recall.

2. Tools and Technologies Used

Python, Pandas, Matplotlib and Scikit-learn.

3. Theory / Description

Diabetes classification is a supervised machine learning problem in which a model predicts whether a record belongs to the diabetic or non-diabetic class. In this practical, the Outcome column is used as the target: 0 represents a negative outcome and 1 represents a positive outcome. The remaining numerical columns are used as input features.

K-Nearest Neighbors Algorithm

K-Nearest Neighbors (KNN) is a supervised, instance-based classification algorithm. It does not create a complex model during training. Instead, it stores the training records and compares a new record with them when a prediction is required.

To classify a new record, KNN calculates its distance from the training records, selects the K closest neighbours and assigns the class that receives the majority of votes. In this practical, K is set to 5. The choice of K can affect the prediction: a very small value may be sensitive to noise, while a very large value may smooth out local patterns.

Feature Scaling and Data Splitting

KNN depends on distance calculations. If one feature has a much larger numerical range than another, it can dominate the distance. StandardScaler is therefore used to standardize the input features. The dataset is split into 80% training data and 20% testing data. The scaler is fitted on the training data and then applied to the test data.

Evaluation Metrics

A confusion matrix summarizes the predictions as True Positive, True Negative, False Positive and False Negative. Accuracy measures the proportion of all correct predictions. Error rate is one minus accuracy. Precision measures how many records predicted as positive are actually positive, while recall measures how many actual positive records are correctly identified. These measures together provide a clearer picture of classifier performance.

4. Procedure

1. Import Pandas, Matplotlib and the required Scikit-learn functions.
2. Load diabetes.csv and inspect the records, dataset shape, data types and missing values.
3. Separate the Outcome column as the target and use the remaining columns as input features.
4. Split the dataset into training and testing sets using an 80:20 ratio.

5. Use StandardScaler to normalize the training and testing features.
6. Create a KNeighborsClassifier with n_neighbors = 5 and train it using the training data.
7. Predict the outcomes for the test data.
8. Calculate the confusion matrix, accuracy, error rate, precision and recall.
9. Plot the confusion matrix using Matplotlib.

5. Working

When the model receives a test record, it compares the standardized feature values with the training records. The five nearest records are selected, and their class labels determine the prediction by majority voting.

The predicted labels are compared with the actual test labels. The confusion matrix is also separated into TN, FP, FN and TP values, which are then used to understand the classifier's correct and incorrect predictions.

6. Applications

- Exploratory analysis of diabetes-related datasets.
- Supporting risk-screening research using patient records.
- Demonstrating classification methods for healthcare data.

7. Result

The KNN classifier was implemented on the diabetes dataset using five neighbours. Its predictions were evaluated using a confusion matrix, accuracy, error rate, precision and recall.

8. Code Explanation

1. The notebook loads diabetes.csv, displays basic dataset information and separates Outcome from the input features.
2. train_test_split() creates the training and testing sets. StandardScaler is fitted on the training features and used to transform both sets.
3. KNeighborsClassifier(n_neighbors=5) creates the classifier. The fit() method trains it and predict() generates predictions for the test records.
4. confusion_matrix() calculates the prediction matrix. The ravel() method extracts TN, FP, FN and TP.
5. Accuracy, error rate, precision and recall are calculated using Scikit-learn metrics, and the confusion matrix is visualized with Matplotlib.

Practical 6: Sales Data Clustering Using K-Means and Hierarchical Clustering

1. Aim

To implement K-Means and Hierarchical Clustering on the sales dataset, and determine the number of clusters using the Elbow Method.

2. Tools and Technologies Used

Python, Pandas, Matplotlib, Scikit-learn and SciPy.

3. Theory / Description

Clustering is an unsupervised machine learning technique used to group data points based on their similarity. Unlike classification, clustering does not require predefined target labels. Records in the same cluster should have similar feature values, while records in different clusters should be relatively dissimilar.

In this practical, sales records are grouped using three numerical features: quantity ordered, price each and sales. StandardScaler is applied before clustering because these features may have different scales. Standardization prevents a feature with a larger numerical range from dominating distance-based grouping.

K-Means Clustering

K-Means is a partition-based clustering algorithm. It divides the data into a specified number of clusters, represented by their centroids. Initially, centroids are selected. Each record is assigned to its nearest centroid, and the centroids are recalculated as the mean of the records assigned to each cluster. Assignment and centroid updates repeat until the clusters stabilize or the stopping condition is reached.

Elbow Method

The Elbow Method helps select a suitable number of clusters. It calculates the Within-Cluster Sum of Squares (WCSS), also called inertia, for different values of K. WCSS measures the total squared distance between each record and its assigned cluster centre. As K increases, WCSS generally decreases. The value of K near the point where the decrease begins to slow down is considered an elbow and may be selected as a practical cluster count.

Hierarchical Clustering

Hierarchical clustering creates a hierarchy of clusters. Agglomerative clustering follows a bottom-up approach: initially, each record forms its own cluster, and the closest clusters are progressively merged. Ward linkage chooses merges that minimize the increase in within-cluster variance. A dendrogram visualizes the merging process and helps inspect the cluster structure.

In this notebook, the dendrogram is generated from the first 100 scaled records for readability, while AgglomerativeClustering with three clusters is applied to the full scaled dataset. The results of K-Means and hierarchical clustering are visualized using scatter plots.

4. Procedure

1. Import Pandas, Matplotlib, StandardScaler, KMeans, AgglomerativeClustering and SciPy hierarchy functions.
2. Load sales_data_sample.csv using the Latin-1 encoding and inspect the dataset.
3. Select QUANTITYORDERED, PRICEEACH and SALES as numerical input features.
4. Standardize the selected features using StandardScaler.
5. Run K-Means for cluster counts from 1 to 10 and store the WCSS value for each run.
6. Plot the Elbow graph and inspect the change in WCSS to choose a suitable number of clusters.
7. Apply K-Means with three clusters and store the assigned labels in the dataset.
8. Generate a Ward-linkage dendrogram using the first 100 scaled records.
9. Apply AgglomerativeClustering with three clusters to the full scaled data and visualize both clustering results.

5. Working

The notebook first selects and standardizes the sales features. K-Means is run for several values of K so that the WCSS values can be plotted. The practical then fits a three-cluster K-Means model and attaches its cluster labels to the original dataset.

For hierarchical clustering, SciPy linkage is used to build a Ward-linkage hierarchy for the first 100 records and the dendrogram displays the merging pattern. Scikit-learn AgglomerativeClustering then assigns three hierarchical clusters to all records. Scatter plots show the cluster assignments for both methods.

6. Applications

- Grouping similar sales records for analysis.
- Identifying sales patterns and segments.
- Supporting business planning, inventory analysis and marketing decisions.

7. Result

K-Means and Hierarchical Clustering were implemented on the selected sales features. The Elbow Method was used to inspect cluster counts, a three-cluster solution was visualized, and hierarchical grouping was examined using a dendrogram and scatter plot.

8. Code Explanation

1. The notebook loads the sales dataset and selects QUANTITYORDERED, PRICEEACH and SALES as the clustering features.
2. StandardScaler transforms the selected features so that they are on comparable scales.
3. A loop fits K-Means models for K values from 1 to 10 and stores each model's inertia in the wcss list. The values are plotted to form the Elbow graph.
4. A K-Means model with three clusters is fitted, and its predicted cluster labels are stored in the KMeans_Cluster column.
5. SciPy linkage() creates a Ward hierarchy from the first 100 scaled records, and dendrogram() plots the hierarchy.
6. AgglomerativeClustering(n_clusters=3, linkage="ward") assigns hierarchical cluster labels to the full dataset. The two sets of labels are visualized using scatter plots.